

torch.functional.norm#

<https://pytorch.org/docs/stable/generated/torch.functional.norm.html>

Description:

Returns the matrix norm or vector norm of a given tensor. `torch.norm` is deprecated and may be removed in a future PyTorch release. Its documentation and behavior may be incorrect, and it is no longer actively maintained. Use `torch.linalg.vector_norm()` when computing vector norms and `torch.linalg.matrix_norm()` when computing matrix norms. For a function with a similar behavior as this one see `torch.linalg.norm()`. Note, however, the signature for these functions is slightly different than the signature for `torch.norm`.

input (Tensor) – The input tensor. Its data type must be either a floating point or complex type. For complex inputs, the norm is calculated using the absolute value of each element. If the input is complex and neither `dtype` nor `out` is specified, the result's data type will be the corresponding floating point type (e.g. float if input is complexfloat).

Function Signature

```
torch.functional.norm(input, p='fro', dim=None,  
keepdim=False, out=None, dtype=None)[source]#
```

Parameters

Code Examples

```
>>> import torch
>>> a = torch.arange(9, dtype= torch.float) - 4
>>> b = a.reshape((3, 3))
>>> torch.norm(a)
tensor(7.7460)
>>> torch.norm(b)
tensor(7.7460)
>>> torch.norm(a, float('inf'))
tensor(4.)
>>> torch.norm(b, float('inf'))
tensor(4.)
>>> c = torch.tensor([[ 1, 2, 3], [-1, 1, 4]] ,
dtype=torch.float)
>>> torch.norm(c, dim=0)
tensor([1.4142, 2.2361, 5.0000])
>>> torch.norm(c, dim=1)
tensor([3.7417, 4.2426])
>>> torch.norm(c, p=1, dim=1)
tensor([6., 6.])
>>> d = torch.arange(8, dtype=torch.float).reshape(2, 2, 2)
>>> torch.norm(d, dim=(1, 2))
tensor([ 3.7417, 11.2250])
>>> torch.norm(d[0, :, :]), torch.norm(d[1, :, :])
(tensor(3.7417), tensor(11.2250))
```

Notes & Warnings

WARNING

Warning `torch.norm` is deprecated and may be removed in a future PyTorch release. Its documentation and behavior may be incorrect, and it is no longer actively maintained. Use `torch.linalg.vector_norm()` when computing vector norms and `torch.linalg.matrix_norm()` when computing matrix norms. For a function with a similar behavior as this one see `torch.linalg.norm()`. Note, however, the signature for these functions is slightly different than the signature for `torch.norm`.

NOTE

Note Even though `p='fro'` supports any number of dimensions, the true mathematical definition of Frobenius norm only applies to tensors with exactly two dimensions. `torch.linalg.matrix_norm()` with `ord='fro'` aligns with the mathematical definition, since it can only be applied across exactly two dimensions.

Detailed Documentation

Documentation

Returns the matrix norm or vector norm of a given tensor.

`torch.norm` is deprecated and may be removed in a future PyTorch release. Its documentation and behavior may be incorrect, and it is no longer actively maintained.

Use `torch.linalg.vector_norm()` when computing vector norms and `torch.linalg.matrix_norm()` when computing matrix norms. For a function with a similar behavior as this one see `torch.linalg.norm()`. Note, however, the signature for these functions is slightly different than the signature for `torch.norm`.

`input (Tensor)` – The input tensor. Its data type must be either a floating point or complex type. For complex inputs, the norm is calculated using the absolute value of each element. If the input is complex and neither `dtype` nor `out` is specified, the result's data type will be the corresponding floating point type (e.g. float if input is complexfloat).

`p (int, float, inf, -inf, 'fro', 'nuc', optional)` –

the order of norm. Default: 'fro' The following norms can be calculated:

$$\text{sum}(\text{abs}(x)^{\text{ord}})^{(1./\text{ord})}$$

The vector norm can be calculated across any number of dimensions. The corresponding dimensions of input are flattened into one dimension, and the norm is calculated on the flattened dimension.

Frobenius norm produces the same result as `p=2` in all cases except when `dim` is a list of three or more dims, in which case Frobenius norm throws an error.

Nuclear norm can only be calculated across exactly two dimensions.

`dim (int, tuple of ints, list of ints, optional)` – Specifies which dimension or dimensions of input to calculate the norm across. If `dim` is `None`, the norm will be calculated across all dimensions of input. If the norm type indicated by `p` does not support the specified number of dimensions, an error will occur.

`keepdim (bool, optional)` – whether the output tensors have `dim` retained or not. Ignored

if `dim = None` and `out = None`. Default: `False`

`out` (Tensor, optional) – the output tensor. Ignored if `dim = None` and `out = None`.

`dtype` (torch.dtype, optional) – the desired data type of returned tensor. If specified, the input tensor is casted to `dtype` while performing the operation. Default: `None`.

Even though `p='fro'` supports any number of dimensions, the true mathematical definition of Frobenius norm only applies to tensors with exactly two dimensions. `torch.linalg.matrix_norm()` with `ord='fro'` aligns with the mathematical definition, since it can only be applied across exactly two dimensions.